OBJECTIVE-C · ПРЕЗЕНТАЦИЯ 1 ИЗ 3

Objective-C: основы языка

История и особенности · способы запуска · данные и переменные · функции и замыкания · поток управления

План презентации

ЧАСТЬ 1

Основы

- Общие сведения: история, особенности, версия
- Как запустить: Docker, онлайн, локально
- Данные и переменные, владение
- Функции, рекурсия, замыкания (блоки)
- Условия, циклы, итераторы

ЧАСТЬ 2

Надёжность и структура

- Исключения, NSError, assert
- Префиксы, заголовки, модули, библиотеки
- Обобщённые типы
- Коллекции Foundation
- Ввод-вывод: консоль и файлы

ЧАСТЬ 3

Модель исполнения

- Конкурентность: NSThread, GCD, NSOperation
- ООП, протоколы, категории, ФП
- Специфические конструкции: сообщения, KVC/KVO
- Среда выполнения: рантайм, objc_msgSend
- Управление памятью: MRC, ARC

История: от препроцессора до iPhone

1981–83	Брэд Кокс и Том Лав знакомятся со Smalltalk-80 и пишут препроцессор **OOPC** — «C с объектами»
1983	Компания **PPI** (позже StepStone) продаёт компилятор Objective-C
1986	Книга Кокса **Object-Oriented Programming: An Evolutionary Approach**
1988	**NeXT** лицензирует язык, добавляет его в GCC; на нём пишут NeXTSTEP
1993–94	Спецификация **OpenStep**; начинается свободная реализация **GNUstep**
1997	Apple покупает NeXT → NeXTSTEP становится Mac OS X и Cocoa
2006–07	**Objective-C 2.0**: свойства, fast enumeration, сборщик мусора (Mac OS X 10.5)
2008	iPhone SDK — язык становится одним из самых популярных в мире
2011–15	**ARC** (2011), литералы (2012), модули (2013), лёгкие generics (2015)
2014 →	Apple представляет **Swift**; Objective-C остаётся в поддержке и в legacy-коде

Отличительные особенности

От C

- **строгое надмножество C**: любой C-код валиден
- указатели, структуры, препроцессор, `printf`
- компиляция в машинный код, без VM
- раздельные `.h` и `.m` файлы

От Smalltalk

- объекты общаются **сообщениями**: `[obj doSomething:x]`
- позднее связывание, тип `id`
- классы — тоже объекты, есть метаклассы
- богатая интроспекция во время выполнения

Новое

- **категории** — методы в чужие классы
- **протоколы** (интерфейсы), блоки (замыкания)
- сообщение `nil` возвращает 0, а не падает
- **нет** namespace, перегрузки, шаблонов C++

Спецификация и версии

- **Стандарта ISO/ANSI нет.** Язык определяется реализацией
- Де-факто описания:
  - Apple, **The Objective-C Programming Language** (Objective-C 2.0)
  - документация clang: Objective-C Literals, ARC, Modules
  - исходники рантаймов: Apple objc4, GNUstep libobjc2
- Последняя «именованная» версия — **Objective-C 2.0**; дальше язык развивался расширениями clang

Компонент	Что используем
Компилятор	clang 18 (LLVM)
Рантайм	GNUstep libobjc2 2.3, ABI `gnustep-2.2`
Foundation	GNUstep Base 1.31.1
GCD	swift-corelibs-libdispatch 6.4
Язык	Objective-C 2.0 + ARC, блоки, литералы, generics, модули

> GCC тоже компилирует Objective-C, но без ARC, блоков и литералов, поэтому берём clang.

Способ 1. Docker

env/Dockerfile

```
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive
COPY install-toolchain.sh /tmp/install-toolchain.sh
RUN chmod +x /tmp/install-toolchain.sh \
 && /tmp/install-toolchain.sh \
 && rm -rf /tmp/objc-build /var/lib/apt/lists/*

WORKDIR /work
CMD ["bash"]
```

```
$ cd 14-objective-c/01/src
$ docker build -t objc-dev env/
$ docker run --rm -v "$PWD":/work objc-dev make run
===== 00_hello_arc
Objective-C = C + Smalltalk
===== 00_hello_mrc
Hello, Objective-C!
...
```

- `install-toolchain.sh` собирает из исходников с фиксированными версиями:
- – libobjc2 → libdispatch → GNUstep Make → GNUstep Base
- Тот же скрипт работает и без Docker

Способ 2. Онлайн-среда



onecompiler.com/objectivec — проверка возможностей компилятора

Сервис	Что внутри
OneCompiler	clang 7.0.1 + GNUstep; литералы — да; **ARC — нет, блоки — нет**; есть stdin
JDoodle	GCC (5.3 … 15.2): классический синтаксис, MRC
Coding Ground (tutorialspoint)	онлайн-IDE к учебнику; GCC + GNUstep

> Подойдут примеры в стиле `00_hello_mrc.m`, `04_ownership_mrc.m`: MRC и без блоков.

Способ 3. Локальное окружение

```
# Linux (Debian/Ubuntu): полный стек с ARC и блоками
$ sudo ./env/install-toolchain.sh

# Linux: быстрый вариант из пакетов (GCC-рантайм, без ARC)
$ sudo apt install gobjc gnustep-devel

# сборка одного файла
$ clang $(gnustep-config --objc-flags) -fobjc-arc \
      hello.m $(gnustep-config --base-libs) -o hello

# macOS: Foundation от Apple
$ clang -fobjc-arc -framework Foundation hello.m -o hello
```

- **Linux** — скрипт `install-toolchain.sh` или пакеты дистрибутива
- **Windows** — WSL2 (Ubuntu) + тот же скрипт; есть и нативные сборки GNUstep для MSVC
- **macOS** — `xcode-select --install`, код примеров совместим
- `gnustep-config` выдаёт флаги компилятора и линковщика

Примитивные типы — из C

01_types.m

```
char       c   = 'A';
int        i   = 42;
long long  big = 9000000000LL;
float      f   = 3.14f;
double     d   = 2.718281828;
BOOL       ok  = YES;          // YES/NO; на x86-64 Linux = signed char
NSInteger  ni  = -7;           // long на 64-битных платформах
NSUInteger nu  = 7;            // unsigned long
CGFloat    cg  = 1.5;          // double на 64 бит

printf("%c %d %lld %.2f %.3f %d %ld %lu %.1f\n",
       c, i, big, f, d, ok, (long)ni, (unsigned long)nu, cg);
printf("sizeof: int=%zu long=%zu NSInteger=%zu BOOL=%zu id=%zu\n",
       sizeof(int), sizeof(long), sizeof(NSInteger), sizeof(BOOL), sizeof(id));
```

```
$ ./01_types
A 42 9000000000 3.14 2.718 1 -7 7 1.5
sizeof: int=4 long=8 NSInteger=8 BOOL=1 id=8
```

Объектные типы и литералы

01_types.m

```m
NSString     *s    = @"Objective-C";              // строковый литерал
NSNumber     *n    = @42;                         // «упакованное» число
NSNumber     *flag = @YES;
NSArray      *arr  = @[@1, @"two", @3.0];         // разнотипный массив
NSDictionary *dict = @{@"lang": s, @"year": @1984};
id    anything = s;            // id — указатель на любой объект
anything = n;                  // ... можно переприсвоить объект другого класса
Class cls  = [s class];        // класс — тоже объект
SEL   sel  = @selector(length);// селектор — «имя метода»

NSLog(@"%@ %@ %@ %@ year=%@", n, flag, arr[1], dict[@"lang"], dict[@"year"]);
NSLog(@"anything=%@ class=%@ selector=%@",
      anything, cls, NSStringFromSelector(sel));
```

```
$ ./01_types
42 1 two Objective-C year=1984
anything=42 class=NSConstantString selector=length
```

Операции над примитивами

01_types.m

```c
int a = 17, b = 5;
printf("%d %d %d %d %d\n", a + b, a - b, a * b, a / b, a % b); // целочисленное деление
printf("%.2f\n", (double)a / b);
printf("and=%d or=%d xor=%d shl=%d shr=%d not=%d\n",
       a & b, a | b, a ^ b, a << 2, a >> 1, ~a);
printf("logic: %d %d %d\n", a > b && b > 0, !(a == b), a != b);

unsigned char byte = 255;
byte++;                                  // переполнение беззнакового: 0
int x = 5;
x += 3; x *= 2; x--;                     // составные операции
printf("byte=%u x=%d\n", byte, x);
```

```
$ ./01_types
22 12 85 3 2
3.40
and=1 or=21 xor=20 shl=68 shr=8 not=-18
logic: 1 1 1
byte=0 x=15
```

- Арифметика, побитовые, логические, составные — как в C
- Целочисленное деление отбрасывает остаток
- Переполнение беззнаковых — по модулю 2^n

Преобразование примитивов

02_casts.m

```
int      i  = 7;
double   d  = i;            // неявное расширение int -> double
double   pi = 3.99;
int      t  = (int)pi;      // явное приведение: дробная часть отбрасывается
int      big = 300;
char     c  = (char)big;    // сужение: 300 mod 256 = 44
unsigned u  = (unsigned)-1; // 4294967295
printf("d=%.1f t=%d c=%d u=%u\n", d, t, c, u);
printf("7/2=%d  (double)7/2=%.1f\n", 7 / 2, (double)7 / 2);
```

```
$ ./02_casts
d=7.0 t=3 c=44 u=4294967295
7/2=3  (double)7/2=3.5
```

- Расширение — неявно и без потерь
- Сужение и `double → int` — с потерей данных
- Явное приведение `(тип)выражение` — как в C

Приведение объектных типов

02_casts.m

```m
id obj = @"I am a string";

if ([obj isKindOfClass:[NSString class]]) {      // проверка во время выполнения
    NSString *str = (NSString *)obj;             // приведение — только для компилятора
    NSLog(@"length=%lu", (unsigned long)str.length);
}

NSArray *wrong = (NSArray *)obj;   // компилятор поверит на слово...
@try {
    NSLog(@"count=%lu", (unsigned long)[wrong count]);
} @catch (NSException *e) {
    NSLog(@"runtime: %@", e.name);  // ...а рантайм — нет
}
```

```
$ ./02_casts
length=13
runtime: NSInvalidArgumentException
```

Приведение `(NSString *)obj` ничего не проверяет и не меняет — оно лишь «успокаивает» компилятор. Проверять нужно в рантайме: `isKindOfClass:`, `respondsToSelector:`.

Области видимости: файл и функция

03_scope.m

```
int        globalCounter = 0;  // глобальная: доступна из других .m через extern
static int fileCounter   = 0;  // static на уровне файла: видна только в этом .m

void tick(void) {
    static int calls = 0;      // static внутри функции: живёт всю программу,
    calls++;                   // но видна только здесь
    fileCounter++;
    globalCounter++;
    printf("calls=%d file=%d global=%d\n", calls, fileCounter, globalCounter);
}
```

```
$ ./03_scope    # tick(); tick();
calls=1 file=1 global=1
calls=2 file=2 global=2
```

Владение: подсчёт ссылок

alloc / new / copy
счётчик = 1, вы — владелец

→

retain
+1 владелец

→

release
−1 владелец

→

счётчик = 0
владельцев нет

→

dealloc
память освобождена

04_ownership_mrc.m

```
Item *it = [[Item alloc] init];  // alloc -> мы владеем, retainCount = 1
[it retain];                     // ещё один владелец: 2
printf("rc=%lu\n", (unsigned long)[it retainCount]);
[it release];                    // 1
[it release];                    // 0 -> dealloc
it = nil;

// Правило: владеем тем, что получили через alloc/new/copy/mutableCopy/retain
NSString *s = [NSString stringWithFormat:@"%d", 5]; // не владеем (autorelease)
Item *tmp = [[[Item alloc] init] autorelease];      // отдадим пулу
printf("before drain: %s\n", [s UTF8String]);
```

```
$ ./04_ownership_mrc
rc=2
Item dealloc
before drain: 5
Item dealloc
after drain
```

ARC: strong и weak в действии

04_ownership.m

```
Person *boss = [[Person alloc] initWithName:@"Boss"];
Person *emp  = [[Person alloc] initWithName:@"Emp"];
emp.boss = boss;              // weak: счётчик ссылок не меняется
boss = nil;                   // последняя strong-ссылка -> dealloc
NSLog(@"emp.boss = %@", emp.boss);   // (null): weak обнулилась
```

```
$ ./04_ownership
+ Boss
+ Emp
- Boss (dealloc)
emp.boss = (null)
```

Передача владения

04_ownership.m

```
@autoreleasepool {
    Person *owner1 = makePerson(@"Data");
    Person *owner2 = owner1;     // теперь два владельца
    owner1 = nil;                // объект жив: его держит owner2
    NSLog(@"still alive: %@", owner2.name);
}                                // конец области — последний release
NSLog(@"after scope");
```

04_ownership.m

```
Person *makePerson(NSString *name) {
    Person *p = [[Person alloc] initWithName:name];
    return p;                    // владение передаётся вызывающему
}
```

```
$ ./04_ownership
+ Data
still alive: Data
- Data (dealloc)
after scope
```

C-функции: вход и выход

05_functions.m

```c
int add(int a, int b) { return a + b; }              // передача по значению

void swap(int *a, int *b) {                          // выходные параметры через указатели
    int t = *a; *a = *b; *b = t;
}

typedef struct { int min; int max; } MinMax;          // «несколько результатов» = struct
MinMax minmax(const int *arr, size_t n) {
    MinMax r = { arr[0], arr[0] };
    for (size_t k = 1; k < n; k++) {
        if (arr[k] < r.min) r.min = arr[k];
        if (arr[k] > r.max) r.max = arr[k];
    }
    return r;
}
```

05_functions.m

```c
int x = 1, y = 2;
swap(&x, &y);
int data[] = {4, -2, 9, 0};
MinMax mm = minmax(data, 4);
printf("add=%d swap: x=%d y=%d min=%d max=%d avg=%.2f\n",
       add(2, 3), x, y, mm.min, mm.max, average(3, 1, 2, 4));
```

```
$ ./05_functions
add=5 swap: x=2 y=1 min=-2 max=9 avg=2.33
```

Методы: сообщения с именованными параметрами

```
[ calc   power:  2    exponent:  10  ] → селектор power:exponent:
  получатель  часть имени  аргумент  часть имени  аргумент   полное имя метода
```

05_functions.m

```objc
@interface Calc : NSObject
+ (NSInteger)sumOf:(NSInteger)a and:(NSInteger)b;            // метод класса (+)
- (double)power:(double)base exponent:(int)e;                // метод экземпляра (-)
- (BOOL)divide:(double)a by:(double)b
        result:(double *)result error:(NSError **)error;     // результат + ошибка
- (NSDictionary *)statsFor:(NSArray<NSNumber *> *)values;    // объект-«кортеж»
@end
```

- `+` — метод класса, `-` — метод экземпляра; имя метода «вплетено» между аргументами
- Типы параметров и результата — в скобках; `NSError **` — выходной параметр ошибки

Рекурсия

06_recursion.m

```c
unsigned long long factorial(unsigned n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}

void hanoi(int n, char from, char to, char via, int *moves) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to, moves);
    (*moves)++;                                  // переносим диск n: from -> to
    hanoi(n - 1, via, to, from, moves);
}
```

```
$ ./06_recursion
20! = 2432902008176640000, hanoi(10) moves = 1023
```

- Базовый случай + рекурсивный вызов
- Глубина ограничена стеком потока
- Хвостовая оптимизация **не гарантируется**

Замыкания: блоки

```
int (^square)(int) = ^int(int x) { return x * x; };
```

`int`	`(^ square )(`	`int`	`) = ^int(int x) {`	`return x * x; };`
результат	имя переменной	параметры	литерал блока	тело

07_blocks.m

```m
int (^square)(int) = ^int(int x) { return x * x; };   // тип ^(аргументы)
typedef NSString *(^Formatter)(NSString *);
Formatter shout = ^(NSString *s) {                    // тип результата выводится
    return [s.uppercaseString stringByAppendingString:@"!"];
};
void (^hello)(void) = ^{ NSLog(@"no args, no result"); };
hello();
NSLog(@"%d %@", square(7), shout(@"blocks"));
```

```
$ ./07_blocks
no args, no result
49 BLOCKS!
```

Захват переменных

07_blocks.m

```
int base = 10;
int (^addBase)(int) = ^(int x) { return x + base; };  // захват ПО ЗНАЧЕНИЮ
base = 100;
printf("addBase(1) = %d\n", addBase(1));              // 11, не 101

__block int counter = 0;                              // захват ПО ССЫЛКЕ
void (^inc)(void) = ^{ counter++; };
inc(); inc(); inc();
printf("counter = %d\n", counter);                    // 3

NSMutableArray *log = [NSMutableArray array];         // объект захвачен strong-ссылкой,
void (^logIt)(NSString *) = ^(NSString *m) { [log addObject:m]; }; // менять его можно
logIt(@"a"); logIt(@"b");
NSLog(@"log = %@", [log componentsJoinedByString:@","]);
```

```
$ ./07_blocks
addBase(1) = 11
counter = 3
log = a,b
```

- Обычные переменные — **копия** на момент создания
- `__block` — общая переменная
- Объекты захватываются **сильной** ссылкой

Условные конструкции

08_control.m

```objc
int t = 23;
if (t < 0) {
    NSLog(@"freezing");
} else if (t < 20) {
    NSLog(@"cool");
} else {
    NSLog(@"warm");
}
NSString *kind  = (t % 2 == 0) ? @"even" : @"odd";   // тернарный оператор
NSString *name  = nil;
NSString *shown = name ?: @"anonymous";              // GNU-расширение «?:»
NSLog(@"%@ %@ | %@ | %@", kind, shown, describe(WeekdaySun), describe(WeekdayTue));
if (name == nil && !name) NSLog(@"nil is false");    // nil, 0, NO — «ложь»
```

```
$ ./08_control
warm
odd anonymous | weekend | workday
nil is false
```

Циклы

08_control.m

```c
for (int i = 0; i < 5; i++) {          // классический for
    if (i == 1) continue;              // пропустить итерацию
    if (i == 4) break;                 // выйти из цикла
    printf("for i=%d\n", i);
}

int n = 3;
while (n > 0) printf("while n=%d\n", n--);

int attempts = 0;
do {                                   // тело выполнится хотя бы раз
    attempts++;
} while (attempts < 0);
printf("do-while attempts=%d\n", attempts);
```

```
$ ./08_control
for i=0
for i=2
for i=3
while n=3
while n=2
while n=1
do-while attempts=1
```

- `for`, `while`, `do-while`, `break`, `continue` — как в C
- Плюс `for-in` для коллекций

Итераторы: for-in и NSEnumerator

09_iterators.m

```
NSArray *langs = @[@"C", @"Smalltalk", @"Objective-C"];
for (NSString *s in langs) printf("%s ", s.UTF8String);       // fast enumeration
printf("\n");

NSDictionary *years = @{@"C": @1972, @"Smalltalk": @1980};
for (NSString *key in [years.allKeys sortedArrayUsingSelector:@selector(compare:)])
    NSLog(@"%@ -> %@", key, years[key]);                      // по словарю — ключи

NSEnumerator *rev = [langs reverseObjectEnumerator];          // явный итератор
id obj;
while ((obj = [rev nextObject])) printf("%s ", [obj UTF8String]);
printf("\n");
```

```
$ ./09_iterators
C Smalltalk Objective-C
C -> 1972
Smalltalk -> 1980
Objective-C Smalltalk C
```

Блоки как конструкция управления

09_iterators.m

```
[langs enumerateObjectsUsingBlock:^(NSString *s, NSUInteger idx, BOOL *stop) {
    NSLog(@"%lu: %@", (unsigned long)idx, s);
    if (idx == 1) *stop = YES;                              // аналог break
}];
[years enumerateKeysAndObjectsUsingBlock:^(id k, id v, BOOL *stop) {
    if ([v intValue] < 1975) NSLog(@"old: %@", k);
}];
```

```
$ ./09_iterators
0: C
1: Smalltalk
old: C
```

- `*stop = YES` — аналог `break`
- `NSEnumerationReverse`, `NSEnumerationConcurrent` — порядок и параллельность

Итоги первой части

- Objective-C = C + объектная модель Smalltalk; формального стандарта нет, ориентир — clang
- Запуск: Docker, онлайн-компиляторы (без ARC), локально на Linux, Windows, macOS
- Примитивы из C, объекты — указатели; `id`, `Class`, `SEL`, литералы `@`
- Владение — подсчёт ссылок; ARC: `strong`, `weak`, `copy`
- Методы — сообщения с именованными частями; ошибки через `NSError **`
- Блоки — замыкания с захватом по значению и `__block`
- Управление — из C, плюс `for-in`, `NSEnumerator`, перебор блоками

Источники

1. Tutorialspoint. Objective-C Tutorial
   https://www.tutorialspoint.com/objective_c/index.htm
2. Feifan Zhou. Learn Objective-C in 24 Days (Binpress)
   https://www.binpress.com/learn-objective-c-24-days/
3. Apple. Programming with Objective-C
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html
4. Apple. Advanced Memory Management Programming Guide
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/MemoryMgmt/Articles/MemoryMgmt.html
5. Apple. Blocks Programming Topics
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Blocks/Articles/00_Introduction.html
6. Clang. Objective-C Automatic Reference Counting
   https://clang.llvm.org/docs/AutomaticReferenceCounting.html
7. GNUstep
   https://gnustep.github.io/
8. OneCompiler: Objective-C online
   https://onecompiler.com/objectivec